Mechanics Sketch Tool

Status report and build plan — 9 October 2026

1 Summary

The sketcher works: steps 1 to 5 of the build plan are done. Geometry can be drawn, constrained, dimensioned and dragged, and editing a dimension re-solves the whole drawing. It runs as a single HTML file that opens straight from disk. Supports, loads, the TikZ and SVG exports and the desktop install are still to come (steps 6 to 8).

The tool is a CAD-style constraint sketcher for truss and beam figures: geometry is drawn freely, constrained and dimensioned afterwards, and figures will export to both TikZ and SVG for use in LaTeX.

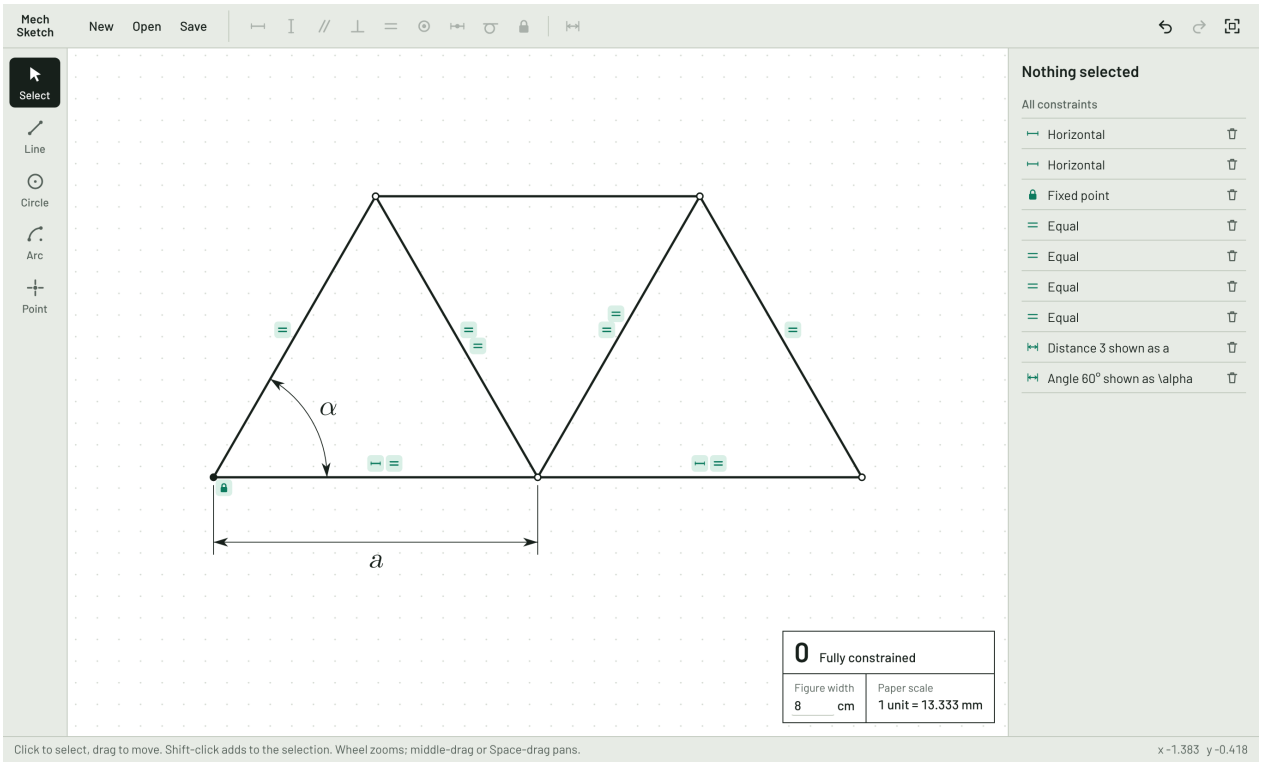


Figure 1: The current build with a Warren truss. One fixed joint, two horizontal chords, equal-length members, one length a and one angle α leave zero degrees of freedom.

2 Decisions

Platform	Browser app, installable as a PWA: one click gives a desktop icon, its own window and offline use.
Stack	TypeScript and Vite. The canvas is drawn directly in SVG.
Solver	FreeCAD's 2D solver `planegcs`, used through its WebAssembly build (`@salusoft89/planegcs` 1.3.0, LGPL). Not written from scratch.
Exports	TikZ and SVG, both first-class, generated from one shared primitive list.
Labels	Typed as LaTeX strings (`F_1`, `\alpha`) and previewed with MathJax.
Hosting	GitHub Pages, needed for the PWA install. The repository does not exist yet (Section 8).

3 Feature scope for the first version

Geometry	Points, lines, circles, arcs.
Constraints	Coincident, horizontal, vertical, parallel, perpendicular, equal length, fixed point.
Dimensions	Distance, angle, radius. Each holds a driving value for the solver and a separate display label such as $2a$, and can be hidden.
Supports	Pin, roller and fixed support, with ground hatching.
Loads	Force arrows, moments, distributed loads.
Editing	Snapping, auto-constraints while drawing (near-horizontal becomes horizontal), dragging of under-constrained geometry, undo and redo.
Feedback	Remaining degrees of freedom; conflicting and redundant constraints are flagged.
Files	Save and load as JSON, SVG export, TikZ export with a "Copy TikZ" button.

Built so far: geometry, constraints, dimensions, editing, feedback, and save and load. Supports, loads and the two exports are not built yet.

Added beyond the agreed list, because the solver provides them directly: point on a line or circle, midpoint, tangent, equal radius, and construction geometry (dashed in the editor, left out of exports).

Out of scope: springs, dampers, masses, velocity and angular velocity arrows, coordinate frames.

4 Direction

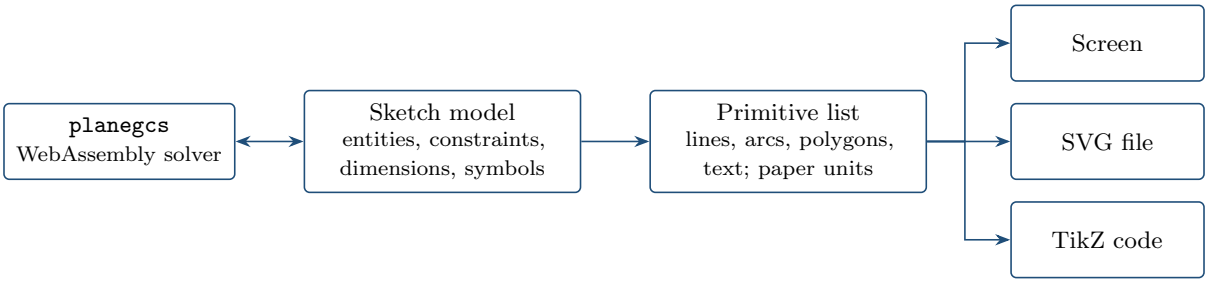


Figure 2: Data flow. Every edit goes through the solver; all three outputs are drawn from the same primitive list.

Four rules shape the code:

1. **One primitive list feeds every output.** Supports, hatching, arrows and dimensions are broken down into lines, arcs, polygons and text once. The screen, the SVG writer and the TikZ writer all read that list, so the exports cannot drift away from what is on screen.
2. **Symbols are sized in paper units.** A pin support is the same few millimetres wide whether the truss is 3 m or 300 mm long. The same holds for arrowheads, line widths and text.
3. **The figure has a set width.** With the width fixed (8 cm, for example), the editor shows labels at their true size relative to the geometry, so anything that does not collide on screen will not collide in the PDF.
4. **Symbols stay out of the solver.** Supports and loads are attached to points and lines and follow them. Only geometry, constraints and dimensions are solved.

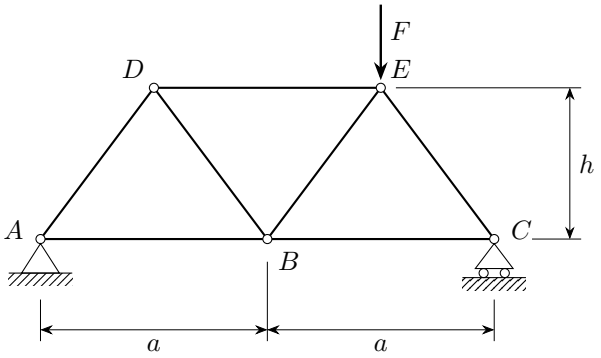


Figure 3: The kind of figure the tool should produce. This one is written by hand; the labels are typeset by LaTeX, so they match the document font.

5 Build plan

	Step	Contents	Status
1	**Scaffold and data model**	Project setup; types for entities, constraints and dimensions; JSON save and load; undo stack. Symbol types come with step 6.	**done**
2	**Solver bridge**	Translate the model to `planegcs` and back. Tested without a user interface on a known truss.	**done**
3	**Canvas and drawing tools**	Pan and zoom; point, line, circle and arc tools; snapping; auto-constraints; select and delete.	**done**
4	**Constraints and dragging**	Constraint toolbar; dragging through the solver; feedback on conflicts and degrees of freedom.	**done**
5	**Dimensions**	Place, edit the value by double-click, set the label, hide or show.	**done**
6	**Symbols and labels**	Supports, hatching, loads, MathJax labels, figure width setting.	not started
7	**Exports**	SVG and TikZ writers. The TikZ output is compiled with LaTeX and compared against the SVG.	not started
8	**PWA and polish**	Install manifest, offline support, icon, keyboard shortcuts, deployment to GitHub Pages.	not started

This build covers steps 1 to 5. The next runnable builds follow step 7 (exports working) and step 8 (installable).

6 What has been checked

- **Automated tests: 33 pass.** They cover the solver bridge (every constraint and dimension type, dragging, conflict and redundancy reports), the model (delete clean-up, JSON round trip), the primitive list (dimension layout, paper scale, picking) and the label preview.
- **Re-solve on a dimension change.** An isosceles truss triangle with base 4 and height 3 solved to joints at $(0, 0)$, $(4, 0)$, $(2, 3)$; changing the values to 6 and 2 gave $(0, 0)$, $(6, 0)$, $(3, 2)$.
- **The app, driven in a browser.** Drawing with snapping, the Warren truss in Figure 1 built from scratch through the interface (the angle of 60° put the top chord at $3 \sin 60° = 2.598$ above the bottom one, as it should), dragging, undo and redo, box selection, delete, save and open, reload restoring the last sketch, and rejecting a conflicting constraint and an over-constraining dimension with a message.
- **The single-file build.** Opened from disk with no server; it drew, dimensioned and restored its sketch after a reload.
- **Speed.** A truss with 77 members (117 entities) re-solves in about 4 ms per drag step.

Not checked: browsers other than Chromium, and touch input.

7 Known limits of this build

- **Colour is all or nothing.** Geometry is blue while the sketch has any freedom left and black once it has none. The solver build does not report which individual joints are still free.
- **Label preview is a stand-in.** Labels such as `F_1`, `2a` and `\alpha` show with sub- and superscripts and Greek letters in a serif font. MathJax replaces this in step 6. The TikZ export will pass labels to LaTeX unchanged either way.
- **Figure width means geometry width.** Dimensions drawn outside the geometry add to the final width.
- **An unanchored sketch drifts.** Dragging one joint can shift the others until a point is fixed.

8 Open items

- **Repository.** The connected GitHub account has one repository, which belongs to a different project. The tool needs its own, and I cannot create repositories from here. The code is committed locally and ready to push. The desktop shortcut (step 8) depends on the repository, because the install needs the app served over HTTPS.
- **Solver licence.** `planegcs` is LGPL. That is fine for a personal tool and for a public repository, provided the licence notice is kept; the README carries it.
- **Name.** "Mech Sketch" is a working title.